Portfolio News Tracker

Engineering Document

AI-Powered Portfolio Intelligence Dashboard for Initialized Capital

1. Problem Statement

This was a take-home assignment to build a portfolio news intelligence system for Initialized Capital. The brief was intentionally open-ended — no prescribed stack, no fixed schema, no defined API contract — to surface judgment in architecture, tooling, and tradeoffs.

The core pipeline required four things:

1	**Portfolio Scraping** Scrape 175 portfolio companies from Initialized Capital's website, preserving sector and description context for downstream query construction.
2	**News Fetching** Fetch recent relevant news per company. Ambiguous names (Cruise, Heap, Envoy) required targeted disambiguation via keyword context.
3	**AI Analysis** Sentiment classification and investor-grade summaries per article and per company, cost-effectively at scale across 175 companies.
4	**Dashboard + Chat** Interactive dashboard with sector filters, signal detection, breaking-news ticker, and an AI chat interface for portfolio Q&A.

Central Challenge

The core challenge was not just building the pipeline — it was making it reliable for 175 companies with ambiguous names, inconsistent news metadata, and varying degrees of public coverage.

2. Tech Stack

Every choice prioritises type safety across the full stack, minimal operational complexity, and single-repository deployability.

2.1 Stack Overview

Layer	Choice	Rationale
Language	TypeScript (strict)	Single language across backend, frontend, and scripts; strict mode catches errors at compile time
Backend Framework	Hono v4.12	faster than Express; native SSE; first-class TypeScript; Cloudflare Workers-compatible
Database	PostgreSQL + Prisma 7	Type-safe ORM; native PrismaPg adapter removes connection pooler requirement; managed on Railway
News — Primary	Exa.ai (neural search)	Semantic search returns fewer irrelevant articles; handles ambiguous company names better than keyword search
News — Fallback	NewsData.io	ToS-compliant keyword fallback when Exa rate limits or results are thin
LLM — Chat	Claude Haiku	Strong multi-turn reasoning for Q&A; prompt caching on stable system prompts reduces repeated-call costs
LLM — Summaries	GPT-5-mini	High output-token volume across 175 companies every 6 hours; frontier models 10-20x more expensive for no quality gain on structured short-form tasks
Frontend	React 18 + Vite + Tailwind + shadcn/ui	SWC compilation; TanStack Query for server state; Framer Motion for transitions
Frontend Scaffolding	Lovable	AI app builder used to generate initial component structure, layouts, and design system; proved useful in designing the overall architecture structure the system was built on
Backend Hosting	Railway (Docker multi-stage)	PostgreSQL co-located with API; auto-deploy from GitHub; Docker pins Node version for deterministic builds
Frontend Hosting	Vercel	Global CDN; zero-config Vite deployment; proxies /api/* to Railway backend
Scheduler	node-cron (in-process)	6-hour refresh cycle; no external queue infrastructure needed at this scale
Cache	In-memory TTL	Zero latency; companies cached 60 s, chat context 30 s; sufficient for single-instance deployment

2.2 LLM Strategy — Task-Appropriate Model Selection

The two models are not in a primary/fallback relationship. They are routed by task type, matching model capability to task complexity.

Claude Haiku

Chat & portfolio briefs

Interactive Q&A requires multi-turn reasoning, coherent synthesis of multiple articles, and consistent voice. Prompt caching on the stable system prompt reduces cost on repeated calls.

GPT-5-mini

Batch summary generation

175 companies x multiple articles x every 6 hours = high output-token volume. Frontier models (Sonnet 4.6, GPT-5.4) cost 10-20x more for no quality improvement on structured classification and short-form summaries.

Key principle: match model capability to task complexity. Over-specifying the model for high-volume structured tasks is a cost error, not a quality gain.

3. System Architecture & Data Flow

Three-tier web application with a scheduled pipeline layer. Vercel (React) → Hono API (Railway) → PostgreSQL. Exa.ai for news; Anthropic (Haiku) for chat; OpenAI (GPT-5-mini) for batch summaries; node-cron triggers the full pipeline every 6 hours; SSE pushes real-time updates to connected frontends.

Stage	What happens
1 — Seed	scrape-portfolio.ts upserts 175 companies with sector + keyword context. Idempotent.
2 — News Fetch	Per-company query (name + sector + keywords). Up to 10 Exa results, filtered for relevance, URL quality, title length, 6-month age, SHA-256 dedup.
3 — Sentiment + Summarise	LLM classifies each article (Positive/Negative/Neutral/Breaking) and generates a 2-3 sentence investor summary. Portfolio briefs in parallel batches of 3.
4 — Serve	Hono returns TTL-cached Prisma results. Frontend renders cards with sparklines, signal badges, sector filters.
5 — Notify	HMAC-SHA256 signed webhooks dispatched; SSE articles.new event triggers React Query invalidation on all connected frontends.

Chat uses Claude tool_use (not MCP) — lookup_company fetches cached DB context; search_news invokes Exa live only when DB context is insufficient. Output is HTML-stripped, PII-redacted, and capped at 2000 chars before returning.

3.2 Data Model & Schema

Parent-child pattern: Company (anchor) → Article and Summary (cascade delete). Webhook is standalone. publishedAt is nullable — never substituted with fetchedAt. urlHash (SHA-256) enforces dedup. Indexes: Company(name, sector), Article(companyId+publishedAt DESC), Article(urlHash), Summary(companyId, generatedAt DESC).

Model	Key fields	Notes
Company	id, name (UNIQUE), sector, keywords (JSON), lastFetchedAt	Upserted by name; keywords drive query disambiguation
Article	id, companyId (FK), urlHash (UNIQUE), publishedAt (nullable★), fetchedAt, sentiment, isBreaking	SHA-256 dedup; publishedAt never substituted
Summary	id, companyId (FK), summaryText, promptVersion, metadata (JSON), generatedAt	promptVersion tracks prompt changes; metadata stores keyThemes/outlook/actionItems
Webhook	id, url (UNIQUE), secret, events, active	HMAC-SHA256 signed; standalone — not per-company

4. Key Engineering Decisions

4.1 Decision Log

Each decision is documented with the chosen option and the reasoning. The focus is on why, not on alternatives that were ruled out.

Decision	Chosen	Rationale
Backend framework	Hono	~10x faster than Express; native SSE; first-class TypeScript
News source	Exa.ai (neural) + NewsData fallback	Semantic search handles ambiguous company names; keyword fallback for thin results
LLM routing	Haiku (chat) / GPT-5-mini (batch)	Frontier models 10-20x more expensive for structured short-form summaries; Haiku reasoning quality matters for Q&A
Parallel fetch	5 concurrent, 2 s between batches	~87 min sequential → ~2 min parallel; 2 s gap prevents Exa rate limits
Date handling	publishedAt nullable	Substituting fetchedAt creates false breaking-news signals; null is honest
Chatbot date context	Stripped from LLM context	Exa metadata unreliable; removing the field is more robust than prompt hedging
Chat tool integration	Direct exa-js + tool_use	stdio MCP is for local clients; direct SDK simpler for server-to-server
Railway deployment	Docker multi-stage	Nixpacks defaulted to Node 22.11; Prisma 7 requires 22.12+

4.2 News Relevance Pipeline

Raw Exa results are never inserted directly. Every article passes through a six-stage filter before reaching the database.

Query construction	Company name paired with sector + signal keywords (e.g. 'Cruise autonomous vehicles funding OR launch'). Filtering happens at query time, not post-hoc.
Relevance filter	Article title or URL must mention the company name or a known keyword alias. Reduces false positives from ambiguous names.
URL-path filter	Homepages, tag pages, search result pages, GitHub repos, /about, /careers, /pricing rejected by regex before any DB write.
Title quality filter	Titles with 2 words or fewer rejected — they typically contain only the company name with no actual news content.
6-month age filter	Articles with publishedAt older than 180 days discarded. Applied at the adapter level because Exa's startPublishedDate is unreliable (see §7).
SHA-256 dedup	URL hashed before insert; createMany with skipDuplicates ensures idempotent batch processing regardless of re-runs.

4.3 Prompting Strategies

Three purpose-built prompts, each with distinct constraints. Shared principles: structured JSON output over free text; sector context in every call; defence-in-depth injection protection; cost optimisation via caching and model routing.

Prompt	Key decisions
Chat (chat.ts)	Role-framed as senior portfolio assistant. Tool priority: lookup_company (DB) before search_news (Exa). “Lead with insight, not setup” suppresses preamble. Dates stripped from context — Exa metadata is unreliable. <user_question> wrapping + 16 regex patterns for injection defence. Temperature 0.2. Prompt caching on system prompt.
Summary (buildSummarySystemPrompt)	12 sector-specific guidance blocks — fintech watches regulatory, security watches CVEs, etc. Structured JSON output: {summary, keyThemes, outlook, actionItems, confidence}. Passes previous outlook for trajectory tracking. 100-word cap enforced in prompt, not post-hoc. Untrusted data warning on article titles.
Sentiment (buildSentimentSystemPrompt)	Company name + sector passed so sentiment is company-relative, not market-wide. Explicit isBreaking thresholds: >$10M raise, >10% layoffs, acquisition, regulatory action. “Add context on business impact” instruction for investor-grade summaries. 280-char limit matches UI card layout.

6. What Worked

What	Why it mattered
Exa neural search	Surfaced relevant results for ambiguous company names with minimal config. Keyword search would have required per-company tuning.
Lovable scaffolding	Initial React component structure, layouts, and design system in place quickly. Proved useful in designing the architecture structure the system was built on.
Task-appropriate LLM routing	GPT-5-mini for batch summaries (10-20x cheaper than frontier models for structured short-form); Haiku for chat where reasoning quality matters. Prompt caching on stable system prompts reduced spend further.
Parallel batch processing	5 concurrent fetches + 2 s inter-batch gap cut full pipeline from ~87 min (sequential) to ~2 min. Gap prevents Exa rate limiting.
Adapter-layer test coverage	76 tests across 13 files, all external calls mocked at the adapter boundary. Fast iteration, no API costs, no flakiness from live services.
Security posture	Token-bucket rate limiting per route, Zod validation on all inputs, 16 anti-jailbreak regex patterns, HMAC-SHA256 webhooks, HSTS + X-Frame-Options headers.

7. Challenges & Fixes

Each issue was identified, diagnosed, and fixed. Documented because the solutions changed the design in non-obvious ways.

Issue	Fix
Exa startPublishedDate is not a hard filter — articles from 2012/2021 appeared despite date params	6-month age filter moved into the Exa adapter, applied post-fetch against the returned publishedAt. API param treated as a hint only.
Substituting fetchedAt for missing publishedAt made old articles appear as breaking news	publishedAt made explicitly nullable. UI handles null with empty states. fetchedAt is used for display ordering only — never substituted into publishedAt.
LLM confidently cited stale or fabricated dates; prompt hedging was insufficient	Date fields removed entirely from LLM context. Model answers on content only. Fewer inputs, fewer hallucinations.
Ambiguous names (Cruise, Heap, Envoy, Amp) pulled unrelated articles	keywords column added to Company; queries built as 'Cruise autonomous vehicles' not 'Cruise'. Keywords require human verification per company.
Nixpacks resolved to Node 22.11; Prisma 7 requires 22.12+, causing silent build failures	Switched to Docker multi-stage build with explicit Node version pin. Deterministic, reproducible builds.
Duplicate private getClient() in exa.ts threw on missing API key, making the Exa client behave inconsistently between the news pipeline and chat search paths	Removed getClient(). Consolidated to a single exported getExa() returning null when unconfigured — consistent null-check pattern across both call sites.
Chat message IDs used Date.now().toString() in ChatWidget.tsx, causing React key collisions when the LLM responded within the same millisecond	Replaced with crypto.randomUUID(). Guarantees globally unique IDs regardless of response timing.

8. What Was Attempted But Not Completed

8.1 Browserbase Integration

Headless browser-as-a-service shortlisted for JS rendering, reliable date extraction, proxy rotation, and reducing Exa dependency. Could not configure — OTP was not delivered during account setup. Dropped in favour of completing the system reliably with Exa.

8.2 Agentic RAG Extension (Designed, Not Built)

Full design produced for a Mastra-based agentic upgrade: live tool-use during chat, pgvector article embeddings, parallel Exa + NewsAPI research workflow, and a dedicated report-writer agent generating cited investment memos on demand. The current chatbot is bounded by the last CRON run; this extension removes that ceiling. Not built within take-home scope.

Component	Role
Mastra Agents (chat + report-writer)	MCP tools + vector RAG for live search and on-demand cited report generation
pgvector (PostgreSQL ext.)	Article embeddings — zero new infra since PostgreSQL is already deployed on Railway
text-embedding-3-small	1536-dim embeddings; 5x cheaper than text-embedding-3-large with comparable retrieval quality
Exa + NewsAPI MCP Servers	HTTP transport on Railway private network; parallel search with time-bounded fallback

9. Future Work

Area	Item	Detail
Infrastructure	Persistent cache (Redis)	Replace in-memory TTL so cache survives restarts and supports horizontal scaling
Infrastructure	Durable job queue (BullMQ)	Move pipeline out of node-cron; adds retries, dead-letter queues, and observability
Infrastructure	OTel tracing	Wire Mastra's native OTel support to Axiom before exposing multi-step report generation
Data quality	Browserbase integration	JS rendering for reliable date extraction and reduced Exa single-source dependency
Data quality	Publisher metadata schema map	Per-domain map of og:published_time / datePublished variants to improve publishedAt coverage
Product	Newsletter delivery	Subscribe form + digest generation designed; needs wiring to SendGrid / Resend
Product	Portfolio comparison view	Cross-company sentiment trends, coverage volume, and signal activity by sector
Product	Alert subscriptions	Per-company webhooks with configurable sentiment thresholds (e.g. 3 consecutive Negative)
Product	Competitor tracking	Extend data model to track competitors alongside portfolio companies

10. Lessons Learned

Nullable is honest	External date metadata cannot be trusted. Design the schema to handle missing dates from day one — substituting fetchedAt creates silent downstream bugs.
Remove bad data from LLM context	Less context produces fewer hallucinations. Removing unreliable fields at source is more robust than prompt-engineering around them.
Neural search needs post-fetch filtering	Exa surfaces relevant results but does not guarantee clean input. Relevance, URL-path, age, and dedup filters are required regardless of API quality claims.
Containerise for deterministic builds	When a framework has a non-obvious runtime requirement (Prisma 7 / Node 22.12+), pin via Docker. Do not trust PaaS defaults to resolve correctly.

Appendix A — API Reference

Method	Endpoint	Auth	Description
GET	/api/health	None	Health check — returns { status, timestamp }
GET	/api/companies	None	All companies with summaries + articles. Supports ?search=, ?sort=name|sector|recent
GET	/api/companies/:id	None	Company detail — full article list, summary, and metadata
POST	/api/refresh	X-Refresh-Token or browser origin	Trigger full pipeline: fetch + sentiment + summarise for all companies
POST	/api/refresh/:companyId	X-Refresh-Token or browser origin	Refresh a single company only
POST	/api/chat	None (rate limited)	Chat with portfolio AI. Body: { message, companyId? }
GET	/api/events	None	SSE stream — pushes articles.new events on pipeline completion
GET	/api/webhooks	X-Refresh-Token	List all registered webhook URLs
POST	/api/webhooks	X-Refresh-Token	Register webhook. Body: { url, secret?, events? }
DELETE	/api/webhooks/:id	X-Refresh-Token	Unregister a webhook by ID

Security Controls

Defence-in-depth — no single mechanism relied upon exclusively.

Control	Implementation
CORS / CSRF	Origin allowlist (localhost:8080 + Vercel); mutation routes validate origin header
Rate limiting	Token bucket per IP — Chat: 10/min · Companies: 60/min · Refresh: 2/10 min
Auth	X-Refresh-Token (timing-safe compare); browser-origin fallback for frontend calls
Input validation	Zod on all endpoints; CUID validation on :id params (400 before DB hit); HTML + control char stripping
Prompt injection	16 regex patterns; <user_question> wrapping; portfolio-only scope enforcement; output PII redaction

Control	Implementation
Headers	HSTS; X-Content-Type-Options; X-Frame-Options; Permissions-Policy; 50 KB body limit
Webhooks	HMAC-SHA256 signed outbound payloads; 30 s default timeout, 300 s for refresh pipeline